

LLM GATEWAY

Gateway protocol reference

The API contract between Claude Code and an LLM gateway: endpoints, headers and body fields to forward, feature degradation when fields are stripped, attribution headers for cost tracking, and model discovery.

This page documents the requests Claude Code sends to a gateway, including the endpoints it calls, the headers and body fields the gateway must forward, and which features stop working when it doesn't. It is written for operators configuring a gateway product to work with Claude Code.

- ❗
 - To roll out an existing or third-party gateway for your organization, see [Roll out an LLM gateway](#)
 - If you're an individual developer authenticating Claude Code to a gateway with a credential you were given, see [Connect Claude Code to an LLM gateway](#)

This page covers:

- **API formats** and the endpoints to serve for each
- **Request headers**: which must reach the upstream and which your gateway can consume
- The **system prompt attribution block** and how it interacts with prompt caching
- **Feature pass-through**: what breaks when headers or body fields are stripped
- **Model discovery**

This page uses two terms for what your gateway does with each header and body field:

- **Forward unchanged**: pass it to the upstream byte-for-byte
- **Consume**: the gateway may read it for routing, attribution, or tracing and need not forward it

Anything not marked forward unchanged is yours to consume or ignore.

API formats

A gateway must expose at least one of the following API formats to Claude Code clients. Which

format Claude Code speaks is determined by the client’s configuration: the variable in the Selected by column of the table below points Claude Code at your gateway in that format.

Format	Selected by	Endpoints	Forward unchanged
Anthropic Messages	ANTHROPIC_BASE_URL	/v1/messages , /v1/messages/count_tokens (optional)	anthropic-beta and anthropic-version request headers
Bedrock InvokeModel	ANTHROPIC_BEDROCK_BASE_URL with CLAUDE_CODE_USE_BEDROCK=1	/model/{model}/invoke , /model/{model}/invoke-with-response-stream	anthropic_beta and anthropic_version request body fields
Vertex rawPredict	ANTHROPIC_VERTEX_BASE_URL with CLAUDE_CODE_USE_VERTEX=1	:rawPredict , :streamRawPredict , count-tokens:rawPredict (optional)	anthropic-beta and anthropic-version request headers, and the anthropic_version request body field

Foundry and Claude Platform on AWS

Microsoft Foundry and the Claude Platform on AWS implement the Anthropic Messages format. Claude Code routes to them through their own variables, ANTHROPIC_FOUNDRY_BASE_URL and ANTHROPIC_AWS_BASE_URL , but a gateway fronting either implements the Anthropic Messages row above. A gateway fronting the Claude Platform on AWS must also forward the anthropic-workspace-id header, which that platform requires on every request.

Optional endpoints and startup traffic

Token-counting endpoints are the only optional ones: when they’re absent, Claude Code estimates context usage locally. Inference requests post to /v1/messages?beta=true , so match on the path, not the full URL. The Vertex method suffixes attach to the publisher model path, as in /projects/{project}/locations/{location}/publishers/anthropic/models/{model}:streamRawPredict .

A gateway also sees best-effort startup traffic it can reject without breaking anything: a HEAD / connectivity probe, and on Bedrock-format gateways a GET /inference-profiles?type=SYSTEM_DEFINED request.

Streaming

Inference responses must stream. Claude Code consumes server-sent events as they arrive, so a gateway that buffers complete responses before relaying them stalls the client.

Format mismatch with the upstream

Which format the client speaks determines what your gateway receives. The common failure mode is a mismatch between the format the client sends to your gateway and the format the upstream provider behind it accepts.

- When the client speaks the Bedrock or Vertex format, Claude Code sends only the subset of its full capability set that those providers accept
- When the client speaks the Anthropic Messages format, Claude Code sends the full set, even if your gateway forwards to a Bedrock or Vertex upstream

Bridging that difference is your gateway's job. [Feature pass-through](#) describes what breaks when it doesn't.

Request headers

Claude Code includes these headers on API requests. Header names are case-insensitive on the wire. Forward `anthropic-version` and `anthropic-beta` unchanged, plus `anthropic-workspace-id` when the upstream is the [Claude Platform on AWS](#); the rest the gateway may consume for routing, attribution, and tracing, and need not forward.

Header	Description
<code>Authorization</code> , <code>x-api-key</code>	The developer's gateway credential, in one or both headers depending on which <u>credential variable</u> they set
<code>anthropic-version</code>	API version, currently <code>2023-06-01</code> . Bedrock- and Vertex-format requests also carry the <code>anthropic_version</code> body field, whose value is the provider dialect string, not this header's value
<code>anthropic-beta</code>	Comma-separated capability values for the request. Forward the header verbatim; do not allowlist individual values, because the set changes with Claude Code releases. When the developer authenticates with a claude.ai login, which is possible when <code>ANTHROPIC_BASE_URL</code> is set without a gateway credential variable, this header also carries an OAuth capability that the upstream requires, and stripping it fails those requests with <code>401</code>
<code>x-claude-code-session-id</code>	A unique identifier for the current Claude Code session. Use it to aggregate all requests from one session without parsing request bodies

Header	Description
<code>x-claude-code-agent-id</code>	Identifier of the subagent that issued the request, present only on requests from an agent Claude Code spawned inside the session. Use it with the session ID to attribute cost to parallel agents
<code>x-claude-code-parent-agent-id</code>	Identifier of the agent that spawned the requesting agent, present only for nested agents

Subagent IDs are generated fresh for each spawn. Teammate agents, the named members of an **agent team**, reuse a stable name-based ID across reconnections. In both cases the ID identifies an agent, not a person or a device, so do not treat the agent ID header as a user identifier.

If your developers set `ANTHROPIC_CUSTOM_HEADERS`, those headers appear on requests as well.

Forward as open lists

Treat the headers and body fields as open lists, not closed ones. Claude Code gains capabilities over releases, and they arrive as new `anthropic-beta` values, new request body fields, and occasionally new `anthropic-*` or `x-claude-code-*` headers.

When forwarding to an Anthropic-format upstream, pass `anthropic-*` request headers and request body fields through unchanged rather than allowlisting the ones you see today. A gateway pinned to an observed list strips the next capability's header or field and breaks it on the release that introduces it.

The exception is a non-Anthropic upstream such as Bedrock or Vertex, where bridging the schema difference is the gateway's job; see [feature pass-through](#).

System prompt attribution block

Claude Code prepends a short attribution block to the system prompt containing the client version and a fingerprint derived from the conversation. The `api.anthropic.com` endpoint strips the block before processing, so it does not affect first-party prompt caching; any other upstream receives it as part of the prompt. Anthropic and the cloud providers' Claude endpoints read it for attribution, so to omit it set `CLAUDE_CODE_ATTRIBUTION_HEADER=0` rather than stripping it in the gateway.

From Claude Code v2.1.181, the block is stable for the lifetime of a conversation when requests route through a custom base URL, so a gateway-side prompt cache keyed on the full request body works without disabling it. Before v2.1.181 the block included a per-request token; on those versions, set `CLAUDE_CODE_ATTRIBUTION_HEADER=0` if your gateway implements such a cache.

Feature pass-through

Claude Code treats an `ANTHROPIC_BASE_URL` gateway as an Anthropic-format endpoint and sends it the beta headers and request body fields it sends to `api.anthropic.com` , except a small set of diagnostics and defaults reserved for direct connections.

Capabilities that add body fields pair them with a beta header, and the pair travels together. A gateway that strips the header while passing the body, or forwards an Anthropic-format body to an upstream with a different schema, produces hard `400` errors; only when both halves are absent together does the feature turn off quietly. A gateway that rewrites or redacts request bodies for content inspection breaks the pairing the same way stripping does, so inspect without modifying. The table notes where a feature deviates from the pairing.

Fine-grained tool streaming is one of the direct-connection defaults: it is off by default whenever requests route through a custom base URL, and a gateway receives it when developers set `CLAUDE_CODE_ENABLE_FINE_GRAINED_TOOL_STREAMING=1` .

Feature	Header and body pair	Symptom when broken	Remediation
<u>Adaptive reasoning</u>	No beta header. Claude Code sends <code>thinking: {"type": "adaptive"}</code> for Claude 4.6 and later, and treats model names it doesn't recognize, such as gateway aliases, as current models that receive the field	<code>400</code> naming the <code>thinking</code> field or the <code>adaptive</code> tag when the upstream model build doesn't accept it	Upgrade the upstream. On Opus 4.6 and Sonnet 4.6, developers can set <code>CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING=1</code> instead
<u>Context management</u>	Context management beta header pairs with the <code>context_management</code> body field	<code>400</code> with <code>Extra inputs are not permitted</code> . Common when a gateway accepts Anthropic-format requests but forwards them to Bedrock	Forward both, or <u><code>CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1</code></u>
<u>Extended context and interleaved thinking</u>	Beta headers only, no body field	Silently unavailable when the header is stripped; the upstream never sees the capability request	Forward <code>anthropic-beta</code> verbatim
Beta <u>tool fields</u>	Tool-related beta headers pair with tool schema fields such as <code>strict</code> and <code>defer_loading</code>	<code>400</code> naming the unrecognized tool schema field when the body passes through without its header	Forward both, or <code>CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1</code>
<u>Effort</u> and <u>structured outputs</u>	The <code>output_config</code> body field carries effort, structured-output format, and	<code>400</code> naming <code>output_config</code> , often <code>Extra inputs are not</code>	Forward the field and its headers together

Feature	Header and body pair	Symptom when broken	Remediation
	task budget settings; each pairs with its own beta header	<code>permitted</code> , on Bedrock and Vertex upstreams	
<u>Token counting</u>	No beta pairing; uses the <code>count_tokens</code> endpoint	Claude Code falls back to estimating context usage locally	Expose the endpoint if you want exact counts

The `ANTHROPIC_DEFAULT_*_MODEL_SUPPORTED_CAPABILITIES` variables declare model capabilities only in the provider configurations: `CLAUDE_CODE_USE_BEDROCK` , `CLAUDE_CODE_USE_VERTEX` , `CLAUDE_CODE_USE_FOUNDRY` , and `CLAUDE_CODE_USE_MANTLE` . They have no effect behind an `ANTHROPIC_BASE_URL` gateway.

Automatic retry and error forwarding

Claude Code retries automatically after some upstream rejections and disables the rejected capability for the rest of the conversation. Rejections of the `thinking` field, of thinking signatures, and of mid-conversation system messages all recover this way. Context management and tool schema field rejections do not retry; those `400` errors reach the developer.

The retry logic matches on the upstream’s error wording, so forward error response bodies unmodified. A gateway that wraps upstream errors in its own envelope breaks the recovery path even when it preserves the status code.

Disable pre-release capabilities

`CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1` stops Claude Code from sending pre-release capabilities and their body fields on every provider, including context management and the beta tool fields. It does not affect adaptive reasoning, which is selected by model rather than by beta, and it never suppresses the OAuth capability that subscription authentication requires.

The set of capabilities Claude Code sends grows over releases. For current beta header strings, see the beta headers reference; test your gateway against new Claude Code releases rather than pinning to an observed list.

Model discovery

When `ANTHROPIC_BASE_URL` points at a gateway that exposes the Anthropic Messages format, Claude Code can query the gateway’s `/v1/models` endpoint at startup and add the returned models

to the `/model` picker.

Developers enable it by setting `CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY=1`, in their own environment or through managed settings. Discovery is off by default so that gateways backed by a shared API key do not surface every model the key can access to every user. This requires Claude Code v2.1.129 or later.

When discovery runs

Discovery applies only to the Anthropic Messages format. It does not run when:

- Any `CLAUDE_CODE_USE_*` provider variable is set, even if `ANTHROPIC_BASE_URL` is also set
- `ANTHROPIC_BASE_URL` is unset or points at `api.anthropic.com`
- Nonessential traffic is disabled, through `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC` or organization policy

Request and response

The request is `GET /v1/models?limit=1000` with a 3-second timeout, and any redirect is treated as failure so the credential cannot leak to a redirect target. A gateway that responds slowly or redirects `/v1/models`, even `http` to `https`, fails discovery silently; serve the endpoint directly at the configured base URL.

The discovery request sends exactly one credential header:

- `ANTHROPIC_AUTH_TOKEN` as a bearer token, when set
- Otherwise the resolved API key, including an `apiKeyHelper` value, in the `x-api-key` header

This differs from inference requests, which send a helper value in both headers. A gateway that authenticates `/v1/models` must accept `x-api-key` for helper deployments. Any headers from `ANTHROPIC_CUSTOM_HEADERS` are included as well.

Claude Code reads `id` and the optional `display_name` from each entry in the response's `data` array, and ignores entries whose `id` doesn't begin with `claude` or `anthropic`:


```
{
  "data": [
    { "id": "claude-sonnet-4-6", "display_name": "Claude Sonnet
4.6" },
    { "id": "claude-opus-4-8" }
  ]
}
```

Picker entries and caching

The picker is the interactive model list that opens when a developer runs `/model` in Claude Code. Each discovered entry is labeled “From gateway” and uses `display_name` when provided. A discovered ID is skipped only when it exactly matches a row already in the picker, or when both the discovered and existing IDs resolve to **Fable**. Built-in rows are keyed on aliases such as `sonnet`, so a discovered ID such as `claude-sonnet-4-6` adds its own “From gateway” row alongside the built-in entry. The `availableModels` **managed setting** bounds what discovery can add.

Results are cached to `~/.claude/cache/gateway-models.json`, or `%USERPROFILE%\claude\cache\gateway-models.json` on Windows, and refreshed on each startup. If the request fails or the gateway does not implement `/v1/models`, the picker falls back to the cached list from the previous startup or to the built-in model list. If your gateway serves Claude models under aliases that don’t match the discovery filter, developers can add those aliases manually with the **model configuration** variables.

Related resources

For the rest of the gateway documentation set and the underlying API references:

- **LLM gateways overview**: what a gateway is and how it interacts with claude.ai subscriptions
- **Roll out an LLM gateway for your organization**: the admin checklist that uses this contract
- **Connect Claude Code to an LLM gateway**: per-developer configuration and the troubleshooting table
- **Beta headers reference**: the current set of `anthropic-beta` values
- **Messages API**: the API format an Anthropic-format gateway implements